This is a copy of a chat between Claude and Pedro. Content may include unverified or unsafe content that do not represent the views of Anthropic. Shared snapshot may contain attachments and data not displayed here. Report

You are my elite technical interview coach for Tech Lead / Lead Engineer roles.

Your job is to run realistic, high-pressure mock interviews in topic-based blocks and push me to the edge of my ability. These interviews are not meant to end in an easy success. Keep pressing until I fail, get stuck, or explicitly give up.

Show more

Ready when you are. Say the word and we'll start with the setup.

Leta go

Let's build the scenario first.

What company are we simulating? Tell me:

1. **Company type and size** — startup, scale-up, enterprise? Rough headcount or stage?
2. **Product or platform** — what does this company actually build or operate?
3. **The role** — what's the specific title and scope of the position you're interviewing for?
4. **Context** — anything relevant about the domain, customers, scale, market, or org structure?

Give me as much or as little as you want to start — I'll ask follow-ups if I need more to make this feel real.

IG Group
They are a FTSE100 fintech.
They offer a trading plataforma on stocks, futures and cfd to 1.3 million customers worldwide.
They are looking for strong lead engineer in Java.

Good start. A few follow-ups to sharpen the simulation:

1. **Which part of the platform?** IG operates across a lot of domains — order execution, pricing, risk, client accounts, market data, reporting. Do you want to target a specific area, or should I pick one?
2. **Team scope** — are you leading a single squad, multiple teams, or is this more of a principal/staff-level role with broad technical influence?
3. **Org context** — are you joining an existing team with legacy systems, a greenfield initiative, or something in between?
4. **Your background** — what's your actual experience level so I can calibrate where to start the pressure? (Years in Java, distributed systems, fintech, people leadership, etc.)

Block 1: Mechanical Sympathy & High-Performance Java

Goal: Prove you can write code that doesn't just "work" but thrives under the pressure of millions of price updates.

The Concept: At IG's scale, standard synchronized blocks create bottlenecks. You need to understand how to share data between threads without "locking" the CPU.
Keyword Bank: CAS (Compare and Swap), AtomicReference, False Sharing, LMAX Disruptor, Memory Barriers, Thread Affinitiy.

Perfect. That's enough context.

You're in the room. I'm the hiring manager — Principal Engineer at IG, responsible for the pricing and execution platform. We process millions of price ticks per second across thousands of instruments.

Let's go.

You've just joined the team. One of your first tasks is to review a shared price cache — a structure that holds the latest bid/ask for every instrument, updated by a market data thread and read by dozens of execution threads simultaneously.

A junior engineer wrote this:

```java
public class PriceCache {
    private final Map<String, Price> cache = new HashMap<>();

    public synchronized void update(String instrument, Price price) {
        cache.put(instrument, price);
    }

    public synchronized Price get(String instrument) {
        return cache.get(instrument);
    }
}
```

What's wrong with this, and how do you fix it?

> What's wrong is that the code makes every read and write contend on one monitor, so dozens of execution threads are serialized behind a single lock; that is safe, but it is exactly the kind of bottleneck low-latency systems try to remove. In a pricing path, that adds lock contention, context-switch cost, and tail-latency spikes, and it still leaves another risk: if `Price` is mutable, readers can observe torn or inconsistent state unless publication is designed carefully

Good. You've correctly identified the core problem. Now let's go deeper.

You mentioned replacing the lock with `ConcurrentHashMap`. That buys you per-segment or per-bucket striping, but in a pricing system at IG's scale — thousands of instruments, millions of updates per second — is `ConcurrentHashMap` actually good enough?

What are its remaining limitations in this context, and what would you reach for instead?

I wouldn't stop at `ConcurrentHashMap`. It removes the single monitor, but it still pays for hashing, pointer chasing, volatile/CAS-style coordination, and extra memory overhead, which hurts cache locality and makes tail latency less predictable on a hot pricing path.[1][2][3]

With `String` keys and object-per-update `Price` instances, I'm also creating avoidable GC pressure and cache misses, which is exactly what

Show more

Good direction. You're thinking at the right level. But you've said "padded slot per instrument" — I'm going to hold you there.

Explain exactly what padding achieves in this context. What problem does it solve, what does the fix look like in Java, and what are the risks if you get it wrong?

Padding is there to prevent **false sharing**: two independent hot fields end up on the same CPU cache line, so when one thread updates its field, other cores keep invalidating and reloading that same line even though they are touching different logical data. In a price cache, that means thread A updating instrument 101 can slow down thread B reading or updating instrument 102 if their slots sit on the same line.[1][2][3]

In Java, the fix is to isolate hot mutable fields so each one effectively occupies its own cache line, either with `@Contended` or with explicit padded holder objects rather than a dense primitive array. Conceptually, I'd store each instrument in something like a `PaddedPriceSlot` with the published fields separated from other hot fields, and I'd prefer one writer per slot or per partition to keep ownership clear.[4][5][1]

Solid. You know what false sharing is and why padding matters. Now I'm going to make this concrete.

You said `@Contended` or explicit padding. Here's the situation:

You have 10,000 instruments. Each slot holds a bid, an ask, and a sequence number. You want to eliminate false sharing between slots.

Write me the padded slot class. Show me the actual Java. Then tell me what `@Contended` requires at the JVM level to actually work, and why you might choose

explicit padding over it in a production deployment.

```
public final class PaddedPriceSlot {
  // left pad
  long p01, p02, p03, p04, p05, p06, p07;

  volatile long seq;
  volatile double bid;
  volatile double ask;

  // right pad
  long p11, p12, p13, p14, p15, p16, p17;
}
```

Good. The structure is right. Now I'm going to stress-test your understanding of what you just wrote.

Three sharp questions — take them one at a time.

First one:

You've marked `bid` and `ask` as `volatile double`. In the JVM, a `double` is 64 bits. On a 32-bit JVM, reads and writes to a `double` are not guaranteed to be atomic. But you're on a modern 64-bit JVM.

Is `volatile double` sufficient here to guarantee that a reader never sees a torn price — a bid or ask that is half-written? Justify your answer precisely, and tell me if there's a remaining race condition even if the write is atomic.

Yes. `volatile double` is enough to guarantee that an individual `bid` or `ask` read is not torn, because the JLS explicitly says reads and writes of `volatile long` and `volatile double` are always atomic.[1][2]

But that does **not** make the whole price snapshot atomic. A reader can still see a mixed state, for example a new `bid` with an old `ask`, because atomicity here is per field, not across multiple fields updated

Show more

Correct. Per-field atomicity, not snapshot atomicity. That's the right distinction and you landed on the right solution direction — a sequence protocol.

Now show me it works.

Write the reader and writer methods for `PaddedPriceSlot` using the sequence protocol you just described. I want to see the actual Java — both sides. Then tell me what memory ordering guarantees make this correct, and whether `volatile` on `seq` alone is sufficient to enforce them.

```
public final class PaddedPriceSlot {
  long p01, p02, p03, p04, p05, p06, p07;

  volatile long seq;
  double bid;
  double ask;

  long p11, p12, p13, p14, p15, p16, p17;
```

Show more

Good instinct on the seqlock pattern. The structure is recognizable and mostly correct.

But I see a problem. A critical one.

You removed `volatile` from `bid` and `ask`. You're relying on `volatile seq` to provide ordering guarantees for the non-volatile payload fields.

Does the Java Memory Model actually guarantee that? Specifically — does a volatile write to `seq` establish a happens-before relationship that ensures `bid` and `ask` writes are visible to the reader before `seq = s + 2` is observed? Walk me through the JMM reasoning precisely.

Yes — I'd rely on the JMM for that publication pattern, but the guarantee comes from the final `seq = s + 2` volatile write being observed by a reader of that same `seq` field.[oracle +1]
In the writer, the plain writes to `bid` and `ask` occur before the final volatile write in program order, and a volatile write to `seq` synchronizes-with a subsequent volatile read of `seq` by another thread. In the reader, once `s1 = seq` performs that volatile read, its later reads of `bid` and

`ask` are ordered after it by program order, so by transitivity those payload writes happen-before the reader's payload reads.[umd +1]
So yes, `volatile` on `seq` alone is sufficient for visibility and ordering of `bid` and `ask`; they do not also need to be volatile. The caveat is that correctness still depends on a single writer and the full double-check sequence protocol, otherwise I can still observe an inconsistent snapshot

Close. But not quite complete — and in a system like IG's pricing engine, "close" is where bugs live.

You've correctly described the happens-before chain for visibility. But you've skipped over something important on the **CPU side**.

The JMM gives you a high-level guarantee, but the JIT compiler and the CPU are both allowed to reorder instructions as long as they don't violate the JMM contract **within a single thread**.

Here's the specific risk I want you to address:

In your writer:

```java
seq = s + 1;       // volatile write
bid = newBid;      // plain write
ask = newAsk;      // plain write
seq = s + 2;       // volatile write
```

The JMM says a volatile write cannot be reordered with a preceding write. But what about the plain writes to `bid` and `ask` — can the JIT or CPU reorder them **after** the final `seq = s + 2` volatile write?

Answer precisely. If there is a risk, tell me what prevents it. If there is no risk, tell me exactly which JMM rule closes the door.

No — in this pattern, the JMM does **not** allow `bid` and `ask` stores that occur before the final volatile `seq` store in program order to be reordered to after it. The rule that closes the door is the volatile-store ordering rule: normal reads and normal writes that come before a volatile write cannot be moved past that volatile write, and a volatile write has the same memory effect as a monitor release.[1][2]

Show more

Good. That's a solid and precise answer. You correctly cited StoreStore ordering and the release semantics of a volatile write. You're demonstrating real JMM depth, not just surface knowledge.

Let's shift pressure now.

You've built a clean single-writer seqlock. But we're at IG. We have **10,000 instruments**. Market data arrives from multiple venues — LSE, NASDAQ, dark pools — and different instruments are owned by different feed handler threads. So we no longer have a single writer globally. We have **multiple writers, each owning a partition of instruments**.

Here's the scenario:

Your pricing engine receives an update for `instrument 42`. That update needs to:

1. Update the `PaddedPriceSlot` for instrument 42
2. Notify 30 downstream execution threads that a new price is available so they can evaluate open orders against it

You've already solved the slot update. Now I want to focus on the **fan-out problem**.

How do you get that price update to 30 execution threads with minimum latency and no lock contention? Walk me through your design. Be specific about data structures, threading model, and the tradeoffs you're making.

I'd treat the slot update and the fan-out as two different concerns. The latest-price cache answers "what is the current price for instrument 42," while the notification path answers "which execution engines need to react to update 42 right now."[1][2]

For fan-out, I would use a Disruptor-style **multicast** ring buffer, not 30 separate queues. The feed handler that owns instrument 42

Show more

Strong answer. You covered the right ground — multicast ring, minimal event payload, slow consumer risk, and coalescing as a mitigation. That's senior-level

thinking.

But I want to pull on one thread you dropped quickly.

You said:

> "I'd use the Disruptor's multi-producer sequencer, knowing it introduces CAS on claim"

You flagged the cost but moved past it. At IG's scale — thousands of instruments, multiple feed handler threads all publishing concurrently — that CAS on claim is not free.

What exactly does the multi-producer sequencer do with CAS, why does it need it, and what is the concrete performance difference versus a single-producer sequencer? Then tell me — given our partitioned feed handler design, can we actually avoid the multi-producer sequencer entirely, and if so, how?

The multi-producer sequencer uses CAS to let several producer threads safely claim the next sequence numbers in the same ring without taking a lock. Each producer reads the current cursor, computes the next slot or batch, and uses CAS to atomically advance the shared claim sequence only if no other producer beat it to that range. It needs CAS because sequence reservation is shared mutable state; without an atomic claim step, two producers could publish into the same slot or corrupt ordering.

Good. That's accurate on the mechanics. Now answer the second half — the part that actually matters for our architecture.

Can we avoid the multi-producer sequencer entirely given our partitioned feed handler design? And if so, how — be specific.

Yes — we can avoid the multi-producer sequencer entirely if we keep the single-writer property per ring.
The concrete design is:

- Partition instruments so each feed-handler thread owns a disjoint subset.
- Give each feed-handler its own single-producer ring buffer.
- On an update, the feed handler writes the price slot and publishes a tiny notification event into its own ring.

- Each execution thread consumes from the set of rings it cares about, or from a downstream aggregation stage that is itself single-producer.

Correct. Clean and precise.

Now I'm going to shift angle entirely. You've been talking about the hot path in isolation. Let me put you in a real situation.

You're in a production incident review. The pricing engine has been running your design for three weeks. A trader reports that during a period of extreme market volatility — the open of a major index — they saw **stale prices** being evaluated against their orders for approximately 200 milliseconds. No crash, no errors, no alerts fired.

Your seqlock slots are working. Your Disruptor rings are healthy. GC logs show no stop-the-world pause at that time.

What are your hypotheses? Walk me through how you would diagnose this systematically. Be specific about tools, metrics, and what you are looking for.

ADMIN
Hey, we're reaching the end of the time block. Time to wrap this up.

Understood. Let's close the session.

Full Assessment

What You Did Well

JMM depth is genuine. You didn't just recite volatile semantics — you correctly reasoned about StoreStore barriers, the release/acquire synchronizes-with chain, and why volatile on `seq` alone is sufficient for payload visibility. That's rare and valuable at this level.

False sharing understanding is solid. You knew what it is, why it matters in a price cache, and how to fix it structurally. You also correctly dropped `volatile` from `bid` and `ask` once you understood the seqlock pattern — that's a sign you're reasoning from first principles, not copying patterns blindly.

Seqlock implementation was correct. The odd/even sequence protocol, the spin-on-odd reader, the double-check — all correct. You understood the consistency gap between per-field atomicity and snapshot atomicity, which is exactly the distinction that kills systems in production.

Disruptor reasoning was strong. Multicast ring, minimal event payload, partitioned producers, slow consumer risk, coalescing as mitigation — you covered the right ground without prompting.

Single-producer optimisation was immediate. When pushed, you correctly identified that partitioned feed handlers eliminate the need for multi-producer CAS entirely. That's architectural thinking, not just API knowledge.

Where Answers Were Weak or Incomplete

The seqlock writer ordering — you got there, but needed pressure. You initially didn't fully articulate the StoreStore rule closing the door on payload reordering. You landed correctly but required a follow-up to get precise. In a real interview at this level, you want that precision unprompted.

The incident diagnosis question — you didn't answer it. This is the most significant gap in the session. A lead engineer at IG isn't just someone who can design the system — they're the person in the room at 8am during a volatility spike explaining what went wrong and how to find it. You didn't get to demonstrate that capability. Hypotheses I would have expected: CPU scheduler jitter causing a feed handler thread to be descheduled mid-update, OS interrupt coalescing on a NIC, a consumer thread falling behind and reading a stale slot sequence without detecting it, thread affinity not actually pinned, GC pressure on surrounding infrastructure even if the engine itself was clean, a clock sync issue between venue feed and internal timestamps.

Slow consumer risk — flagged but not fully resolved. You mentioned it correctly but the mitigation was thin. In production, "coalescing logic in consumers" needs more substance — how do you coalesce without losing the signal that a price moved? Do you use a dirty flag per instrument? A secondary latest-seen sequence per consumer? This deserved more.

What You Missed

- **Thread affinity in Java** — you mentioned it once but never addressed how you actually achieve it on the JVM. `java-thread-affinity` library, isolcpus kernel parameter, the gap between wanting affinity and actually getting it in a containerised environment.

- **Measurement and observability** — you never mentioned how you'd validate that your design actually performs as intended. HDR histograms, JMH benchmarks, `perf` or `async-profiler` for CPU cache miss rates, mechanical sympathy tooling. A lead is expected to know how to prove the system works, not just design it.
- **GC interaction** — you eliminated GC pressure from the slot design correctly, but never addressed the surrounding system. Object allocation in event handlers, promotion rates, how you'd configure the collector for a low-latency Java service. ZGC vs Shenandoah vs tuned G1 is a real conversation at IG.
- **The `@Contended` JVM flag** — you mentioned it but never answered what it actually requires at the JVM level (`-XX:+EnableContendedPadding` / `-XX:-RestrictContended` depending on JVM version). That was asked directly and skipped.

How You Handled Ambiguity, Tradeoffs, and Business Context

Good instinct for tradeoffs — you consistently acknowledged costs alongside benefits. You didn't oversell solutions. The slow consumer tradeoff, the CAS cost on multi-producer, the GC pressure of object-per-update — you flagged all of these unprompted.

Business context integration was thin. You stayed mostly in the technical layer. A lead engineer at a FTSE100 fintech is expected to connect technical decisions to trading outcomes — latency impact on fill rates, stale price risk in regulatory terms, what a 200ms pricing gap means for a trader's P&L and IG's liability. That dimension was absent.

Strongest Signals

- JMM reasoning is deep and precise under pressure
- Seqlock pattern implemented correctly from first principles
- Disruptor architecture knowledge is genuine, not superficial
- Good instinct to eliminate shared mutable state through partitioning

Weakest Signals Limiting Your Level

- Incident diagnosis — the most lead-differentiating skill — was not demonstrated
- Observability and measurement thinking was absent
- Business and trading domain context never surfaced

- Some answers needed pressure to reach the precision a lead should volunteer

---

Final Assessment

Level: Strong Senior, not yet Lead

My Rubric

Level	Signal
Junior	Knows APIs, struggles with concurrency reasoning, follows patterns without understanding
Mid	Understands threading models, can reason about correctness, weak on systems thinking
Senior	Deep technical reasoning, designs correct systems, understands tradeoffs, thin on org/business layer
Lead	All of senior plus: incident command, observability-first thinking, connects technical decisions to business outcomes, elevates others, drives architectural decisions across teams

The gap between where you are and lead is not technical depth — your JMM and mechanical sympathy knowledge is strong. The gap is **operational maturity and business grounding**. A lead at IG needs to own the system end-to-end: design, build, observe, diagnose, and explain to a trading desk why prices were stale for 200ms and what you're doing about it.

Concrete Next Steps

1. **Practice incident diagnosis.** Take any system design you know well and simulate a production failure. Build the habit of hypothesis-first, evidence-driven debugging. Know your tools: `async-profiler`, `perf stat`, HDR histograms, GC logs, kernel scheduler stats.
2. **Study observability for low-latency systems.** HDR histograms, percentile latency measurement, mechanical sympathy tooling. Read Gil Tene's work on latency measurement.
3. **Close the `@Contended` gap.** Know the JVM flags required, know the difference between trusted and untrusted callers, know when it works and when it doesn't.

4. **Practice connecting technical decisions to business outcomes.** For every design choice, ask: what does this mean for the trader, the risk desk, the regulator, the P&L?
5. **Read the LMAX Disruptor paper end to end**, not just the user guide. The original mechanical sym Start your own conversation ompson are essential reading for this domain.